

NovaCart Account Dashboard — System Design Document

Project: NovaCart Account Dashboard **Repository:** `teamTree` **Program:** HC&D Associates Capstone — App Developer + App Consultant **Document version:** 1.0 **Date:** 2026-08-06 **Status:** Describes the system as implemented on `main` at commit `ac19db3`

1. Purpose and Scope

1.1 Purpose

The NovaCart Account Dashboard is an internal analytics web application for NovaCart account managers. It reads the **Gold layer** produced by the Data Engineering capstone and presents revenue, order, product, customer, and geographic performance through an interactive dashboard, a set of drill-down insight views, and a natural-language analytics assistant.

This document describes the system's architecture, components, data model, interfaces, deployment topology, cross-cutting concerns, and known limitations. It is the technical companion to the requirements document (`Requirements Document Team Tree Associates Induction July 20th.docx`).

1.2 In scope

- Read-only analytics over the NovaCart Gold data layer
- Three primary dashboard views (Orders, Products, Customers)
- Drill-down insight modals for month, city, product, and customer
- Natural-language query assistant backed by a local LLM with tool/function calling
- CSV export of all displayed datasets
- Multi-currency display with live FX conversion, and UI localization in 10 languages
- Light/dark theming
- Containerized deployment to Snowpark Container Services (SPCS)

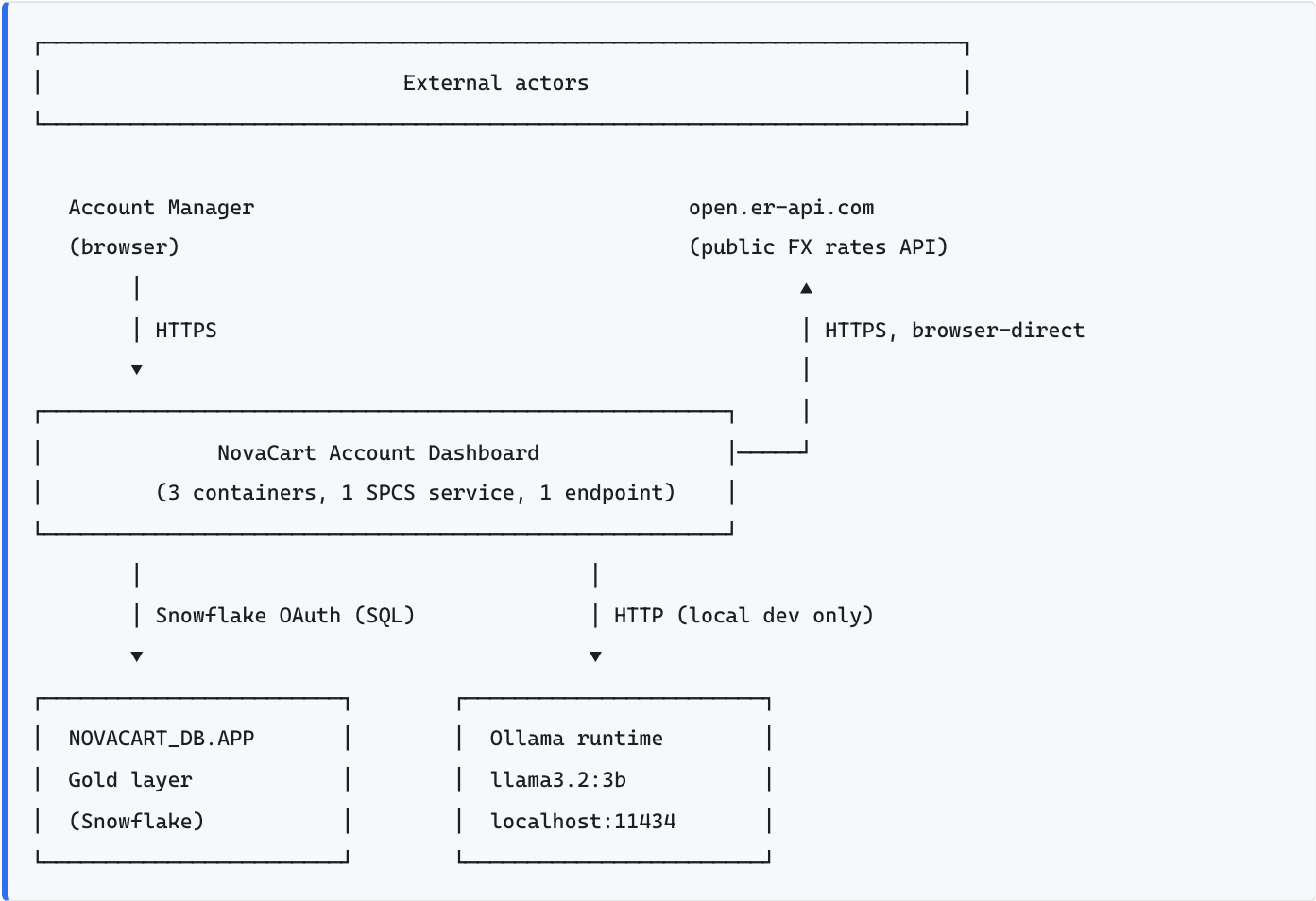
1.3 Out of scope

- Write operations of any kind — the system never mutates the Gold layer
- User management, role-based authorization, or per-franchise data isolation
- ETL / data ingestion (owned by the Data Engineering capstone)
- Multi-tenancy beyond the per-group SPCS service isolation used by the program
- Mobile-native clients (the web UI is responsive, but there is no native app)

1.4 Goals

Goal	How it is met
Zero-setup local development	SQLite Gold database ships in the repo; <code>DATA_BACKEND=sqlite</code> is the default
Identical code path local and in production	A single <code>get_connection()</code> / <code>execute_query()</code> abstraction hides the backend
No credentials in the deployed artifact	SPCS mounts a Snowflake OAuth token at <code>/snowflake/session/token</code>
Single public endpoint	An NGINX router container fronts both the SPA and the API on one port
Reproducible deploys	GitHub Actions builds, pushes, and recreates the SPCS service from a spec

2. System Context



Upstream dependency. The Gold layer schema is owned by the Data Engineering team. The dashboard treats it as a stable read-only contract; a schema change there is a breaking change here.

Third-party dependencies. The browser calls `open.er-api.com` directly for FX rates, and Google Fonts for the IBM Plex Sans webfont. Neither is proxied through the backend.

3. Architecture

3.1 Style

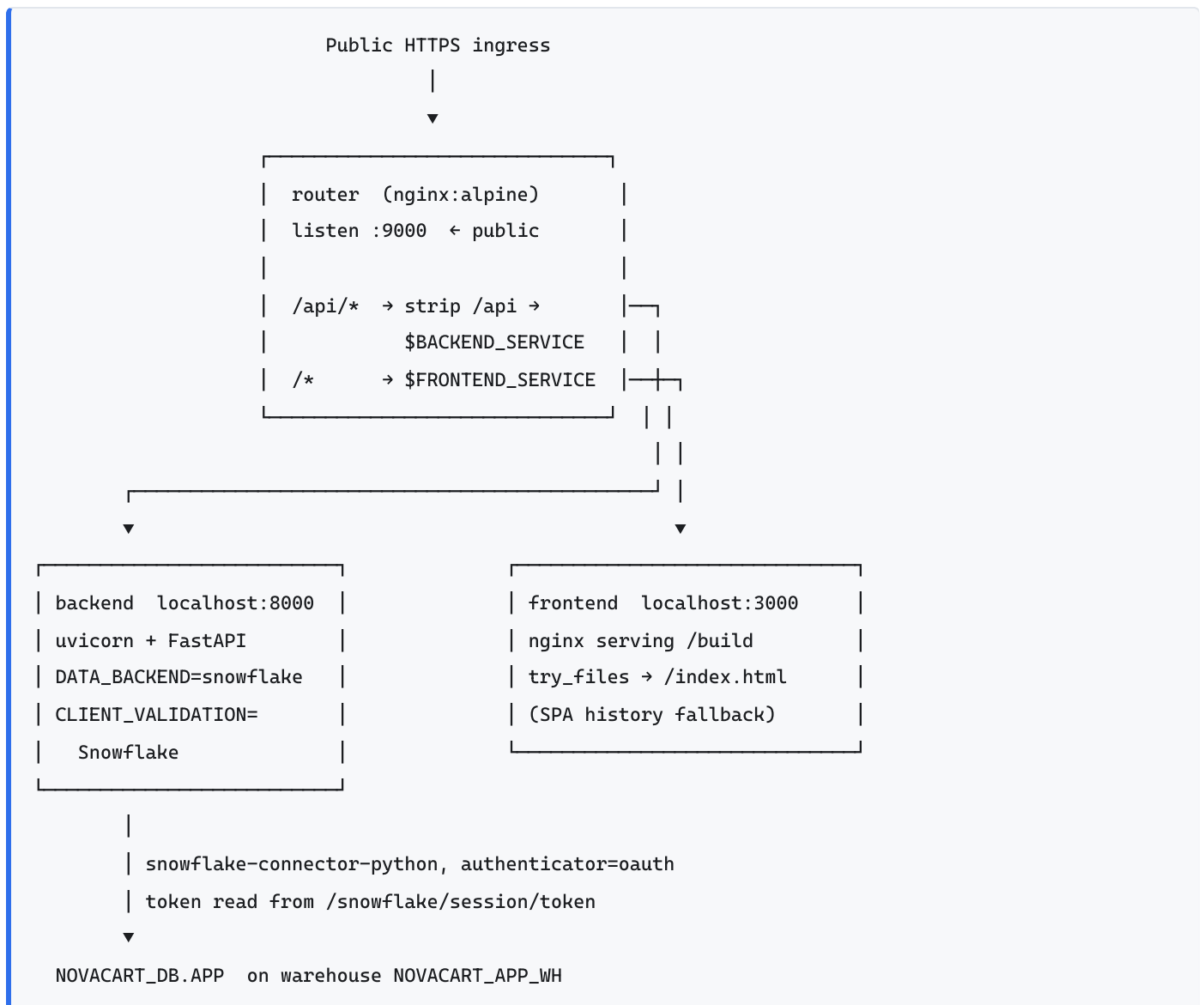
A conventional **three-tier layered architecture** packaged as a **multi-container single-service unit**:

1. **Presentation tier** — React 18 SPA, statically served by NGINX
2. **Application tier** — Python 3.11 / FastAPI REST API; all business logic and SQL
3. **Data tier** — SQLite (local) or Snowflake (deployed), selected at runtime

A fourth container, the **router**, is infrastructure rather than a tier: it collapses the SPA and the API behind a single public port so the deployment needs exactly one SPCS ingress endpoint and no CORS configuration.

3.2 Deployed topology (SPCS)

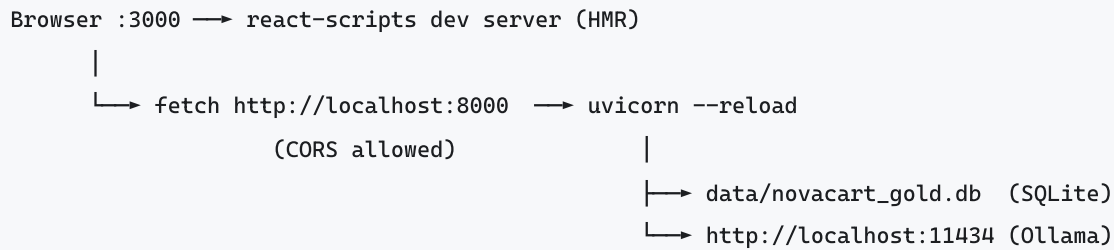
All three containers run in **one** SPCS service (`FRONTEND_SERVICE_GROUP<N>`) and therefore share a network namespace — they address each other over `localhost` .



Service spec facts (from [deploy-group.yml](#)):

- Compute pool: `NOVACART_BACKEND_POOL`
- `MIN_INSTANCES=1`, `MAX_INSTANCES=1` — **single instance, no horizontal scaling**
- Public endpoint `routerendpoint` on port 9000
- Endpoint usage granted to `NOVACART_EXT_ROLE`

3.3 Local development topology



The router is **not** used locally. The two environments therefore differ in three meaningful ways — API base URL, CORS enforcement, and data backend — all controlled by environment variables (§8.1).

3.4 Request lifecycle

Example: user opens the Orders view in the deployed app.

1. Browser `GET /` → router → frontend container → `index.html` + JS bundle.
 2. React Router redirects `/` → `/orders`; `OrdersView` mounts.
 3. `useEffect` fires `loadData()`, which issues three **parallel** requests via `Promise.all`:
`/api/franchise/summary`, `/api/franchise/orders`, `/api/franchise/cities`.
 4. Router rewrites `/api/franchise/summary?...` → `/franchise/summary?...` and proxies to `localhost:8000`.
 5. FastAPI handler calls `get_connection()` → new Snowflake connection → `execute_query()` → one aggregate SQL statement → rows normalized to lowercase-keyed dicts.
 6. Handler rounds monetary values to 2dp and returns JSON.
 7. React stores results in component state; Recharts renders; `CurrencyContext.fmt()` converts USD → selected currency and formats per locale at render time.
 8. In parallel and independently, `ServiceStatus` polls `/api/health` every 30 s.
-

4. Component Design

4.1 Backend — backend/

File	Responsibility
main.py	FastAPI app, CORS, all HTTP route handlers, all analytics SQL
connection.py	Backend selection, connection construction, query execution + row normalization
ai_assistant.py	LLM tool schema, system prompt, tool dispatch, chart-type suggestion
test_ai.py	Manual smoke script against a running /chat
Dockerfile	python:3.11-slim, installs requirements-snowflake.txt, runs uvicorn

Design choices.

- *No ORM, no service layer.* Handlers own their SQL directly. For a fixed set of read-only aggregate endpoints this removes a layer with no offsetting benefit, and keeps each endpoint's cost visible in one place.
- *Aggregation pushed into SQL.* Every endpoint returns pre-aggregated rows — at most 20 rows for customers, 10 for products, 12 for monthly series. Python does formatting only. This keeps payloads small and lets the warehouse do the work.
- *Filters composed, not templated.* Endpoints that accept optional city / state build a where_clauses list and a parallel params list, then join with AND. All user-supplied values are bound parameters; only fixed SQL fragments are concatenated.
- *Connection per request, no pool.* get_connection() constructs a fresh connection on every call. Acceptable for SQLite; a known cost driver against Snowflake (\$11.1).

Endpoint groups.

- System — /health
- Auth — /authorize
- Franchise — the five list/summary endpoints backing the three main views
- Insights — two per-entity drill-down endpoints, each issuing 3–4 queries
- AI Assistant — POST /chat

4.2 Data access layer — connection.py

Two constructors behind one façade:

```

get_connection()          # dispatches on DATA_BACKEND
└─ get_sqlite_connection() # row_factory = sqlite3.Row
└─ get_snowflake_connection()
    └─ if Path("/snowflake/session/token").exists(): # SPCS
        |     authenticator="oauth", token=<mounted token>
    └─ else: # local
        |     keypair auth from SNOWFLAKE_PRIVATE_KEY_PATH (PEM → DER PKCS8)

```

`execute_query(conn, sql, params)` returns `list[dict]` from either backend, lowercasing Snowflake's uppercase column names so handler code is backend-agnostic.

The SPCS-vs-local branch inside `get_snowflake_connection()` is what allows the same image to run in both places: presence of the mounted token file is the signal, so no environment flag has to be kept in sync.

Portability caveat. The abstraction is not currently complete — see §11.2. All SQL in the repo is written against SQLite dialect and placeholder syntax.

4.3 Router — `router/`

An `nginx:alpine` image whose config is a template rendered at container start:

```

CMD envsubst '$FRONTEND_SERVICE $BACKEND_SERVICE' \
    < /nginx.conf.template > /etc/nginx/nginx.conf && nginx -g 'daemon off;'

```

Two locations only:

Location	Behavior
<code>/api</code>	<code>rewrite /api/(.*) /\$1 break;</code> then <code>proxy_pass http://\$BACKEND_SERVICE/</code>
<code>/</code>	<code>proxy_pass http://\$FRONTEND_SERVICE/</code>

Rendering at start rather than build time means one image serves any group and any container naming, driven purely by service-spec env vars.

4.4 Frontend — `frontend/src/`

Stack: React 18, react-router-dom 6, Recharts 2, Create React App (`react-scripts` 5). No state library, no CSS framework, no TypeScript.

Composition (`App.js`):


```

ThemeProvider
└─ CurrencyProvider      (currency + language + FX rates + formatters + t)
  └─ DateRangeProvider   (global start/end date)
    └─ BrowserRouter
      ├── /              → redirect /orders
      ├── /orders        → OrdersView
      ├── /products      → ProductsView
      ├── /customers     → CustomersView
      ├── *              → redirect /orders
      └─ ChatWidget       (outside Routes – persists across navigation)

```

Pages.

Page	Data sources	Interactions
OrdersView.js	<code>summary</code> , <code>orders</code> , <code>cities</code> (parallel)	3 stat cards; monthly revenue bar chart → click opens <code>MonthlyInsightsModal</code> ; horizontal revenue-by-city chart (top 10) → click opens <code>CityInsightsModal</code> ; 2 CSV exports
ProductsView.js	<code>products</code>	Top-10 horizontal bar chart; detail table; row click → <code>InsightsModal</code> via <code>/products/{id}/insights</code> ; CSV export
CustomersView.js	<code>customers</code>	Client-side sortable 5-column table (numeric and string aware); row click → <code>InsightsModal</code> via <code>/customers/{id}/insights</code> ; CSV export

Note that drill-downs are of two kinds. Product and customer drill-downs call dedicated `/insights` endpoints. Month and city drill-downs **re-issue the existing list endpoints** with a narrowed date range or a city/state filter and compose the result client-side — `handleBarClick` fires four parallel requests, `handleCityClick` three. This reuses the filter parameters already present on the franchise endpoints instead of adding two more endpoints, at the cost of more round trips per drill-down.

Components.

Component	Role
<code>Navbar</code>	Tab navigation, <code>ServiceStatus</code> , language selector (10), searchable currency selector (~160 ISO 4217 codes), combined "export all"
<code>ServiceStatus</code>	Polls <code>/health</code> every 30 s; renders a healthy/degraded/offline/checking dot
<code>DateRangePicker</code>	Popover with 8 presets + custom from/to; stages to local temp state, commits on Apply
<code>ChatWidget</code>	Floating assistant; normal/expanded sizes; conversation history; renders <code>DynamicChart</code> for structured replies
<code>DynamicChart</code>	Renders <code>bar</code> <code>line</code> <code>stats</code> <code>table</code> from an arbitrary AI result payload, auto-detecting keys
<code>InsightsModal</code>	Product/customer drill-down: summary tiles, monthly trend, top related entities, category breakdown
<code>MonthlyInsightsModal</code> , <code>CityInsightsModal</code>	Composed drill-downs for a month / a city
<code>EmptyState</code> , <code>ErrorBanner</code>	Uniform no-data and error presentation

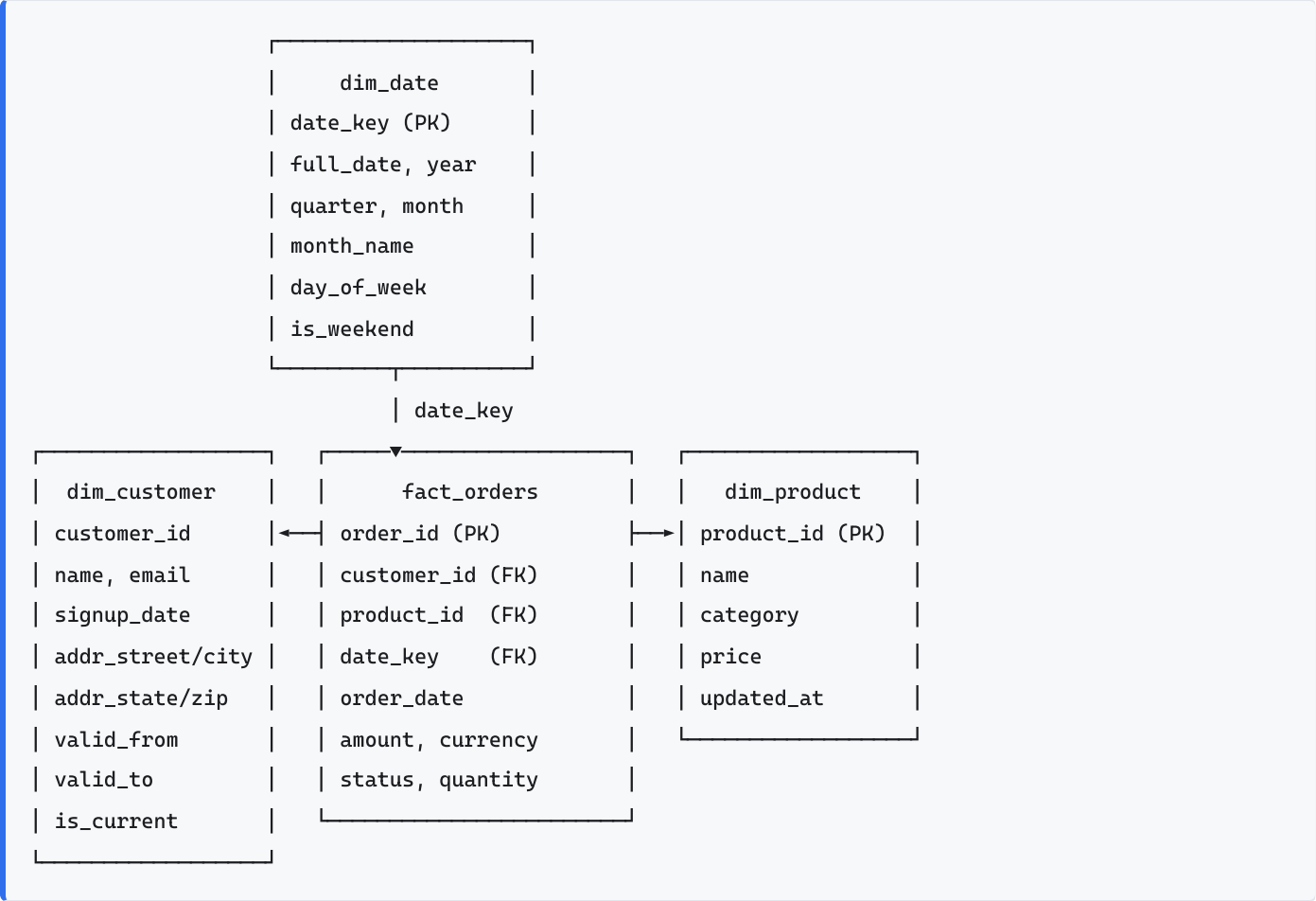
State strategy. Three React contexts hold cross-cutting UI state; everything else is local `useState` in the page that owns it. There is deliberately **no client-side data cache** — each page fetches on mount, so tab switches re-fetch. Simple and always fresh; see §11.1 for the cost.

`DateRangePicker.onApply(start, end)` passes the new dates as arguments rather than relying on context having updated, because `setStartDate` / `setEndDate` do not take effect until the next render. Pages use `loadData(start = startDate, end = endDate)` so both call styles work.

5. Data Design

5.1 Gold layer schema

Star schema, one fact and three dimensions. Local SQLite copy is 3.1 MB and contains 30,000 orders, 400 customers, and 15 products.



`dim_customer` is a **Type-2 slowly changing dimension** (`valid_from`, `valid_to`, `is_current`). Every query that joins it must filter `is_current = 1`, otherwise historical address rows fan out the join and inflate revenue. This is enforced by convention in each handler, not by a view or a shared helper — a maintenance risk (§11.4).

5.2 Business rules encoded in the query layer

Rule	Implementation	Applies to
Revenue counts only fulfilled orders	<code>f.status IN ('delivered', 'shipped')</code>	every revenue/order metric
Customer attributes use current address	<code>c.is_current = 1</code>	every <code>dim_customer</code> join
Date filtering is inclusive on both ends	<code>f.order_date BETWEEN ? AND ?</code>	all endpoints
Order counts are distinct	<code>COUNT(DISTINCT f.order_id)</code>	all order counts
Money is rounded at the API boundary	<code>round(value or 0, 2)</code>	all monetary fields
Month keys are zero-padded <code>YYYY-MM</code>	<code>d.year '-' printf('%02d', d.month)</code>	monthly series
Products list is top 10 by revenue	<code>ORDER BY revenue DESC LIMIT 10</code>	<code>/franchise/products</code>
Customers list is top 20 by spend	<code>ORDER BY total_spent DESC LIMIT 20</code>	<code>/franchise/customers</code>

`NULL`-safe aggregation (`row["revenue"] or 0`) is applied consistently so an empty filter result returns zeros rather than `null`.

5.3 Data volume and shape

Payloads are small and bounded by design:

Endpoint	Rows	Approx. size
<code>/franchise/summary</code>	1 object	< 1 KB
<code>/franchise/orders</code>	≤ 12 (one per month in range)	~1 KB
<code>/franchise/products</code>	10	~1.5 KB
<code>/franchise/customers</code>	20	~3 KB
<code>/franchise/cities</code>	unbounded — all cities in range	~10–40 KB
<code>/franchise/*/insights</code>	~20–30 across 3–4 sections	~5 KB

`/franchise/cities` is the only unbounded response. The UI slices to the top 10 for the chart but downloads the whole set, which is intentional — the CSV export is meant to be complete.

6. API Specification

Base URL: `http://localhost:8000` (local) · `/api` (deployed). All endpoints are `GET` except `/chat`. All responses are JSON. No authentication is enforced at the application layer (§9.1).

Common query parameters, where accepted:

Param	Type	Default	Notes
<code>start</code>	<code>YYYY-MM-DD</code>	<code>2022-01-01</code>	inclusive
<code>end</code>	<code>YYYY-MM-DD</code>	<code>2022-12-31</code>	inclusive
<code>city</code>	string	<code>null</code>	matched exactly against <code>addr_city</code>
<code>state</code>	string	<code>null</code>	matched exactly against <code>addr_state</code>

6.1 System

`GET /health`

```
{ "status": "healthy", "uptime_s": 1204, "backend": "sqlite",  
  "database": { "status": "connected" } }
```

Executes `SELECT 1` to prove the data tier is reachable. Returns **503** with `status: "degraded"` and the exception message if it is not. Consumed by `ServiceStatus`.

6.2 Auth

`GET /authorize`

- `CLIENT_VALIDATION=Dev` → `{ "user": "dev_user", "status": "authorized" }`
- `CLIENT_VALIDATION=Snowflake` → reads the `Sf-Context-Current-User` header injected by SPCS; **422** if absent.

The frontend exports an `authorize()` client function but no component calls it. The endpoint is therefore currently unused by the UI (§11.3).

6.3 Franchise

Endpoint	Params	Returns
GET /franchise/summary	start, end, city, state	{ total_revenue, total_orders, unique_customers, date_range: { start, end } }
GET /franchise/orders	start, end	[{ month, month_name, order_count, revenue }] ascending by month
GET /franchise/products	start, end, city, state	[{ product_id, name, category, units_sold, revenue }] top 10
GET /franchise/customers	start, end, city, state	[{ customer_id, name, city, state, total_orders, total_spent }] top 20
GET /franchise/cities	start, end	[{ city, state, order_count, revenue }] all, desc by revenue

Note the asymmetry: `orders` and `cities` do **not** accept `city / state`. `cities` groups by location so a location filter would be redundant; `orders` has no such reason and this is a genuine gap — the month drill-down in `OrdersView` cannot be combined with a city filter.

6.4 Insights

GET /franchise/products/{product_id}/insights — start, end. Four queries. **404** if the product does not exist.

```
{ "product": { "product_id", "name", "category", "price" },
  "summary": { "total_orders", "total_units", "total_revenue", "avg_order_value" },
  "monthly_trend": [ { "month", "month_name", "order_count", "units_sold", "revenue" } ],
  "top_customers": [ { "customer_id", "name", "city", "state",
                       "order_count", "units_purchased", "total_spent" } ] }
```

GET /franchise/customers/{customer_id}/insights — start, end. Four queries. **404** if the customer has no `is_current = 1` row.

```
{ "customer": { "customer_id", "name", "email", "city", "state" },
  "summary": { "total_orders", "total_items", "total_spent", "avg_order_value",
               "first_order_date", "last_order_date" },
  "monthly_trend": [ { "month", "month_name", "order_count",
                       "items_purchased", "total_spent" } ],
  "top_products": [ { "product_id", "name", "category", "order_count",
                      "units_purchased", "total_spent" } ],
  "category_breakdown": [ { "category", "order_count",
                             "units_purchased", "total_spent" } ] }
```

6.5 AI Assistant

POST /chat

```
// request
{ "message": "Show me top 10 products in 2022", "conversation": [ /* prior turns */ ] }

// response
{ "message": "...natural-language summary...",
  "data": [ /* tool result: array or object */ ],
  "chart_type": "bar",
  "conversation": [ /* full history, to be echoed back next turn */ ],
  "tool_used": "query_top_products",
  "error": null }
```

Conversation state is **client-held**: the server is stateless and the browser round-trips the full message list. This keeps the backend horizontally scalable and needs no session store, at the cost of a request payload that grows with the conversation.

Errors are returned as **200** with a populated `error` field rather than as HTTP error codes, so a failed query still renders as a chat message. The `/chat` handler wraps `process_query` in a broad `try/except` and never raises.

6.6 Error contract

FastAPI's default shape is used throughout:

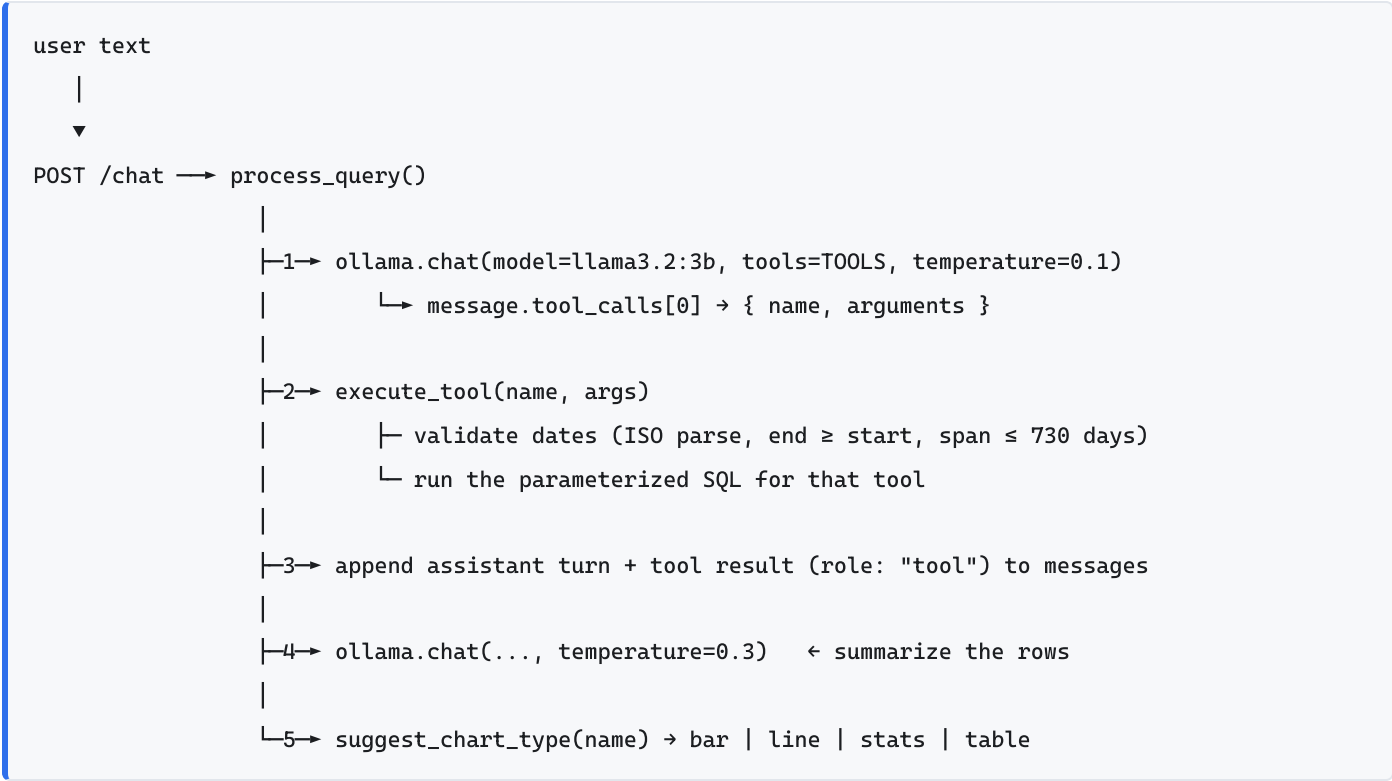
```
{ "detail": "Product not found" }
```

`apiFetch` in `api.js` reads `detail`, falls back to `API error <status>`, and throws an `Error`. Pages catch it into an `error` state that `ErrorBanner` renders. Drill-down failures use `alert()` instead — inconsistent, and worth unifying.

7. AI Assistant Subsystem

7.1 Design

A **tool-calling (function-calling) architecture** rather than text-to-SQL. The model never emits SQL; it selects one of five parameterized tools, and the backend executes a hand-written, parameterized query for that tool.



7.2 Tool catalogue

Tool	Required	Optional	Chart
query_top_products	start_date , end_date	limit (≤50), city , state	bar
query_top_customers	start_date , end_date	limit (≤50), city , state	bar
query_revenue_trend	start_date , end_date , granularity	city , state	line
query_city_breakdown	start_date , end_date	limit (≤50)	bar
query_summary_stats	start_date , end_date	city , state	stats

`granularity` is an enum of `monthly` | `quarterly`, resolved in code by an `if monthly / else quarterly` branch so an out-of-enum value degrades to quarterly rather than injecting anything.

7.3 Why this shape

- **The LLM never touches SQL.** Injection surface is limited to five typed parameter sets. `city` and `state` are bound parameters; `limit` is coerced via `int(min(value, 50))` before interpolation; the grouping/format fragments are chosen from fixed literals. A hallucinated query is impossible — only a wrong *choice* of a known-good query is.
- **A 3B model is sufficient** because the hard part (SQL correctness) is not the model's job. `temperature=0.1` for tool selection maximizes determinism; `0.3` for the summary allows readable prose.
- **The system prompt is defensive** about the one thing the model reliably gets wrong: omitting dates. It states the data range (2022-01-01 to 2022-12-31), gives explicit quarter/half mappings, and repeats "NEVER omit dates" three times. `execute_tool` independently validates dates and returns a structured error rather than trusting it.
- **Local inference, no external LLM calls.** No analytics data leaves the host during local development. This is also the assistant's main deployment limitation (\$11.5).

7.4 Chart rendering

`DynamicChart` receives an arbitrary `data` payload plus a `chart_type` and adapts: `stats` renders known summary keys as tiles; `table` derives columns from `Object.keys(data[0])`; `bar` / `line` auto-detect the category and value keys. This decouples the widget from any particular tool's result shape, so adding a sixth tool needs no frontend change beyond a `suggest_chart_type` entry.

8. Cross-Cutting Concerns

8.1 Configuration

Backend (`.env` , or SPCS spec env):

Variable	Local	SPCS	Purpose
<code>DATA_BACKEND</code>	<code>sqlite</code>	<code>snowflake</code>	selects the data tier
<code>SQLITE_PATH</code>	<code>../data/novacart_gold.db</code>	—	relative to <code>backend/</code>
<code>CLIENT_VALIDATION</code>	<code>Dev</code>	<code>Snowflake</code>	enables CORS + mock auth
<code>PORT</code>	<code>8000</code>	<code>8000</code>	uvicorn bind port
<code>SNOWFLAKE_ACCOUNT</code> / <code>_HOST</code> / <code>_DATABASE</code> / <code>_SCHEMA</code> / <code>_WAREHOUSE</code>	—	set in spec	connection targets
<code>SNOWFLAKE_USERNAME</code> / <code>_ROLE</code> / <code>_PRIVATE_KEY_PATH</code>	optional	not needed	local keypair auth only

Frontend (build-time — CRA inlines `REACT_APP_*` into the bundle):

Variable	Local	SPCS
<code>REACT_APP_BACKEND_URL</code>	<code>http://localhost:8000</code>	<code>/api</code>
<code>REACT_APP_CLIENT_VALIDATION</code>	<code>Dev</code>	<code>Snowflake</code>

`REACT_APP_BACKEND_URL` is baked in at `npm run build` via Docker `ARG / ENV` , so the frontend image is environment-specific. Setting it as a runtime env var on the container (as the SPCS spec also does) has no effect on an already-built bundle — a real footgun, though harmless here because both places use `/api` .

Secrets: `.env` , `rsa_key.p8` , and `rsa_key.pub` are all gitignored; only `.env.example` files are tracked. The Snowflake private key reaches CI through the `SNOWFLAKE_PRIVATE_KEY` GitHub secret.

8.2 Authentication and authorization

- **SPCS ingress** authenticates the caller against Snowflake before the request reaches the router, and injects `Sf-Context-Current-User`.
- **Data-tier authorization** is the real control: the service runs as a Snowflake role with read access to `NOVACART_DB.APP` and nothing more. Endpoint usage is granted to `NOVACART_EXT_ROLE`.
- **Application-layer authorization does not exist.** Every authenticated user sees all data. There is no franchise scoping despite the `/franchise/*` path prefix, and the endpoints take no franchise identifier.

8.3 CORS

Enabled only when `CLIENT_VALIDATION=Dev`, restricted to `http://localhost:3000` and `:3001`, methods `GET` and `POST`. In SPCS the router makes everything same-origin, so no CORS middleware is registered at all — the correct outcome, achieved by making the deployment topology do the work rather than widening a policy.

8.4 Internationalization and currency

Two independent axes, both persisted to `localStorage`:

- **Language** (`nc-language`) — 10 UI languages via `translations.js`; `getTranslations(locale)` resolves with a language-prefix match then falls back to `en-US`.
- **Currency** (`nc-currency`) — ~160 ISO 4217 codes. `CurrencyProvider` fetches `open.er-api.com/v6/latest/USD` once on mount and silently falls back to `{ USD: 1 }` on failure. `fmt()` and `fmtShort()` multiply the stored USD value by the rate and format with `Intl.NumberFormat` under a locale derived from `CURRENCY_LOCALE_MAP`.

Decoupling the two lets a German-speaking manager view figures in USD, or vice versa. Conversion is **display-only** — no converted value is ever persisted or sent to the API.

The backend returns raw `amount` values and the frontend treats them as USD. The `fact_orders.currency` column is not read anywhere. If the Gold layer ever contains non-USD rows, revenue will be summed across mixed currencies (\$11.6).

8.5 Theming

CSS custom properties on `:root` and `[data-theme="dark"]` in `App.css`. `ThemeProvider` sets `data-theme` on `document.documentElement` and persists to `localStorage` under `nc-theme`. Recharts tick colors cannot read CSS variables, so pages compute `tickColor` from the `dark` boolean in JS — the one place the theme is duplicated outside CSS.

8.6 Observability

Present:

- Startup log (port, data backend, validation mode)
- `/health` with a live database probe, polled every 30 s and surfaced in the navbar
- `uptime_s` since process start
- Unicorn access logs to stdout, collected by SPCS

Absent: structured logging, request IDs, latency or error-rate metrics, tracing, and alerting. Debugging a production issue currently means reading container logs.

8.7 Export

`exportCsv.js` builds CSV client-side from already-fetched rows — no export endpoint, no server-side generation.

`exportCsv` writes one dataset; `exportCsvCombined` writes several labelled sections into one file, separated by blank lines, used by the navbar's "export all". Values are escaped for commas, quotes, and newlines; line endings are CRLF for Excel. Download is a `Blob` + object URL + synthetic click, revoked immediately after.

9. Security Considerations

9.1 Threat model summary

Threat	Mitigation	Residual risk
SQL injection via API params	All user values are bound parameters; only fixed fragments concatenated	Low
SQL injection via the LLM	Model cannot emit SQL; only picks from 5 fixed tools; <code>limit</code> integer-coerced	Low
Credential exposure in the image	SPCS mounts an OAuth token at runtime; no secrets in the image or repo	Low
Unauthorized data access	SPCS ingress auth + a read-only Snowflake role	Medium — no per-user or per-franchise scoping
Data exfiltration via CSV	None — any authenticated user can export every dataset	Medium (accepted for an internal tool)
Unbounded query cost / DoS	Date span capped at 730 days in the AI path only	Medium — REST endpoints have no span cap
Third-party script/API compromise	None — Google Fonts and <code>open.er-api.com</code> are loaded directly, no SRI, no CSP	Low-Medium
Sensitive data in error responses	<code>/health</code> returns raw exception text; <code>/chat</code> returns raw error strings	Low-Medium

9.2 Notable specifics

- **PII exposure.** `/franchise/customers/{id}/insights` returns customer `email`, and the customer list returns names and locations. Any authenticated user can retrieve and export these. Given the internal audience this is accepted, but it means the app's blast radius is the full customer contact list.
 - **No rate limiting** anywhere. A single instance with no connection pool is easy to saturate.
 - `/health` **leaks exception text** (`str(e)`) on failure, which may include connection strings or host names. Worth reducing to a generic message with details logged only.
 - **No CSP or Subresource Integrity** on the served HTML.
-

10. Deployment

10.1 Images

Image	Base	Build notes
<code>backend_service_image_group<N></code>	<code>python:3.11-slim</code>	installs <code>requirements-snowflake.txt</code> (FastAPI, uvicorn, dotenv, ollama, snowflake-connector, cryptography)
<code>frontend_service_image_group<N></code>	multi-stage <code>node:18-slim</code> → <code>nginx:alpine</code>	<code>npm run build</code> , static output copied into NGINX; SPA history fallback
<code>router_service_image_group<N></code>	<code>nginx:alpine</code> + <code>bash</code>	config rendered from template at start

All builds are `--platform linux/amd64`, required by SPCS compute pools.

10.2 Pipelines

Workflow	Trigger	Action
prewarm.yml	manual, <code>group_number</code>	builds and pushes starter images so later deploys hit warm registry layers
deploy-group.yml	manual, <code>group_number</code> + <code>fork_url</code>	checks out this repo (router, scripts) and the group's fork (app code); builds all three images with a timestamp tag plus <code>latest</code> ; drops and recreates the SPCS service from an inline spec; grants endpoint usage; polls up to 10× 30 s for the ingress URL via get_url.py
suspend-group.yml	manual	<code>ALTER SERVICE ... SUSPEND</code> to release compute
resume-group.yml	manual	<code>RESUME</code> , wait 90 s, report the URL

[build-and-push.sh](#) is the manual equivalent of the build half, for developers pushing from a laptop with `snow` configured.

10.3 Release strategy

Deploy is `DROP SERVICE` + `CREATE SERVICE`, not a rolling update. Consequences:

- Downtime on every deploy, roughly 2+ minutes (the workflow sleeps 120 s before polling)
- No rollback path in the pipeline — recovery means re-running the deploy against an earlier commit
- No health gate: the workflow reports success once an ingress URL appears, without checking `/health`

Images are tagged with both a build timestamp and `latest`, so a specific prior image does remain addressable in the registry for a manual recovery.

Known inconsistency. `deploy-group.yml` creates a single service named `FRONTEND_SERVICE_GROUP<N>` containing all three containers, but `suspend-group.yml` and `resume-group.yml` target `backend_service_group<N>` **and** `frontend_service_group<N>`. The `backend_service_group<N>` statement matches nothing and is silently skipped by `IF EXISTS` (§11.7).

11. Known Limitations and Risks

Ordered roughly by impact.

11.1 Performance: no caching, no pooling

Every page mount re-fetches; every request opens a new database connection. Against Snowflake, connection setup and warehouse resume dominate latency for queries that themselves scan only 30k rows. A month drill-down issues four such requests in parallel, a city drill-down three. With `MAX_INSTANCES=1` there is no horizontal headroom.

Mitigations, in order of value: a module-level connection (or pool) reused across requests; a short-TTL response cache keyed on the query parameters; client-side caching of results per date range.

11.2 The Snowflake path is not exercised

The abstraction in `connection.py` is sound, but the SQL above it is SQLite-specific, and three concrete defects would surface the first time `DATA_BACKEND=snowflake` is used:

1. `execute_query` references `snowflake.connector.DictCursor`, but `snowflake.connector` is imported **inside** `get_snowflake_connection()` and is not in module scope → `NameError`.
2. All SQL uses `?` placeholders. `snowflake-connector-python` defaults to `pyformat` (`%s`).
3. Monthly keys use `printf('%02d', d.month)`, a SQLite function with no Snowflake equivalent (`TO_CHAR` / `LPAD` would be needed).

Mitigation: hoist the import, introduce a placeholder-style indirection (or move to a dialect-aware query builder), and replace `printf` with a portable expression. Then run the endpoint suite against Snowflake before the next deploy.

11.3 Auth is defined but unused

`/authorize` is implemented on both sides of the wire, but no React component calls `authorize()`. The signed-in Snowflake identity is never read, displayed, or used for scoping, so the app cannot attribute an action to a user or restrict data by user.

11.4 Business rules are duplicated, not centralized

`status IN ('delivered', 'shipped')` appears in roughly a dozen places across `main.py` and `ai_assistant.py`; `is_current = 1` in about eight. A change to either rule requires finding every site, and a missed `is_current = 1` silently inflates revenue rather than erroring.

Mitigation: module-level constants for the predicates, or a Gold-layer view that pre-applies them.

11.5 The AI assistant cannot work as deployed

`ai_assistant.py` calls `ollama.chat(...)`, which targets `localhost:11434`. The backend container ships no Ollama runtime and the SPCS spec provisions none, so in the deployed environment every `/chat` call fails and the widget renders the "Make sure the backend is running and Ollama is started" message. The feature is effectively local-development-only today.

Options: add an Ollama sidecar container with a GPU compute pool; or switch to Snowflake Cortex, keeping the same tool-calling structure; or point at a hosted model API via an SPCS external access integration. The tool-calling design is portable across all three — only `process_query`'s transport changes.

11.6 Currency handling assumes USD

`fact_orders.currency` is never read. Revenue is summed across whatever currencies the fact table holds and then presented as USD before FX conversion. Correct only while the Gold layer is USD-only, and silently wrong the moment it is not.

Mitigation: either assert USD-only at the query layer, or normalize to a reporting currency in SQL using a rate table.

11.7 Suspend/resume workflows target stale service names

See §10.3. The suspend workflow reports success while leaving the service running, if naming ever diverges further. Both files should target `FRONTEND_SERVICE_GROUP<N>` only, or the deploy should be split back into separate services.

11.8 Date presets do not match the data

`DateRangePicker` presets ("Last 7 Days", "This Month", "Year to Date") are computed from the current date, but the Gold layer contains only 2022. Every relative preset returns an empty result and renders `EmptyState`. Only "All Time (2022)" and manual custom ranges produce data.

Mitigation: derive preset anchors from `/franchise/summary`'s `date_range` rather than `new Date()`.

11.9 Smaller items

- **No automated tests.** `test_ai.py` is a manual smoke script; there are no unit, integration, or frontend tests, and no test job in CI.
 - **No REST-side query-cost guard.** The 730-day span limit exists only in `execute_tool`; the REST endpoints accept any range.
 - `/franchise/cities` **is unbounded** — the only response whose size grows with the data.
 - **Inconsistent error UX** — page loads use `ErrorBanner`, drill-downs use `alert()`.
 - **Inline styles and hover logic in JSX.** The table row hover effects in `ProductsView` / `CustomersView` imperatively mutate `style` on the row and every cell; a CSS class would be shorter and cheaper.
 - `useEffect(..., [])` in all three pages omits its dependencies. Correct today only because `DateRangePicker` passes new dates explicitly to `onApply`; fragile if another caller mutates the date context.
 - **Duplicated** `BACKEND_URL`. `ChatWidget` reads `process.env.REACT_APP_BACKEND_URL` directly instead of going through `api.js`, so `/chat` is the one call that bypasses the API client and its error handling.
 - **Dead CSS.** `.page` declares `animation` twice; the second declaration wins and `page-fade-out` is identical to `page-fade-in`.
 - `fmtShort` **formats sub-\$1000 values as \$0K**.
 - **Stray** `frontend/frontend/package-lock.json` — an artifact of running `npm` from the wrong directory; safe to delete.
-

12. Future Enhancements

#	Enhancement	Rationale	Rough effort
1	Fix the Snowflake data path and run the endpoint suite against it (§11.2)	The deployed configuration is currently untested	S
2	Connection reuse + short-TTL response cache (§11.1)	Largest latency and cost win available	S–M
3	Centralize the revenue/SCD predicates (§11.4)	Removes the highest-likelihood source of silently wrong numbers	S
4	Move the assistant to Cortex or an Ollama sidecar (§11.5)	Makes the flagship feature work in production	M
5	Wire <code>/authorize</code> into the UI; display the signed-in user	Prerequisite for any per-user behavior	S
6	Data-driven date presets (§11.8)	Most presets currently return nothing	S
7	Test suite + CI job (pytest for endpoints, RTL for pages)	Nothing currently guards a regression	M
8	Per-franchise scoping — a franchise identifier through the API and a row-access policy	The <code>/franchise/*</code> prefix promises isolation the system does not provide	M–L
9	Rolling deploy with a <code>/health</code> gate and a rollback step	Removes deploy downtime and the manual recovery path	M
10	Structured logging, request IDs, latency/error metrics	Production issues are currently only diagnosable from raw logs	M
11	Server-side pagination for <code>/franchise/cities</code> and the top-N lists	Removes the one unbounded response and enables "show more"	M
12	Persist user preferences (currency, language, theme, default range) server-side	<code>localStorage</code> does not follow a user across devices	M

13. Appendix

13.1 Repository layout

```
teamTree/
├─ backend/                    FastAPI application
│  ├─ main.py                 routes + analytics SQL
│  ├─ connection.py           SQLite / Snowflake abstraction
│  ├─ ai_assistant.py          LLM tool-calling assistant
│  ├─ test_ai.py               manual /chat smoke script
│  ├─ requirements.txt          core deps
│  ├─ requirements-snowflake.txt core + snowflake connector + cryptography
│  ├─ Dockerfile
│  ├─ AI_SETUP.md, QUICK_START.md, TINYLLAMA_SETUP.md
│  └─ .env.example
├─ frontend/                   React 18 SPA
│  ├─ src/
│  │  ├─ App.js                provider composition + routes
│  │  ├─ pages/                 OrdersView, ProductsView, CustomersView
│  │  ├─ components/            Navbar, ChatWidget, DynamicChart, modals, pickers
│  │  ├─ utils/                 api, ThemeContext, DateRangeContext,
│  │  │                           CurrencyContext, translations, exportCsv
│  │  └─ App.css                design tokens + layout
│  ├─ nginx.conf               SPA history fallback
│  ├─ Dockerfile               multi-stage node → nginx
│  └─ .env.example
├─ router/                     NGINX reverse proxy
│  ├─ nginx.conf.template      /api → backend, / → frontend
│  └─ Dockerfile               envsubst at container start
├─ data/
│  └─ novacart_gold.db          3.1 MB SQLite Gold copy - 30k orders, 400 customers, 15 products
├─ .github/
│  ├─ workflows/               prewarm, deploy-group, suspend-group, resume-group
│  └─ scripts/get_url.py        extracts the SPCS ingress URL from `SHOW ENDPOINTS` JSON
├─ build-and-push.sh           manual build + push to the Snowflake image repository
├─ README.md
├─ FRONTEND_AI_TESTING.md
└─ SYSTEM_DESIGN.md            this document
```

13.2 Technology summary

Layer	Technology	Version
Frontend framework	React	^18.2.0
Routing	react-router-dom	^6.21.0
Charts	Recharts	^2.10.1
Build	react-scripts (CRA)	5.0.1
API framework	FastAPI	0.111.0
ASGI server	uvicorn[standard]	0.30.1
Warehouse driver	snowflake-connector-python	3.11.0
Crypto (keypair auth)	cryptography	42.0.8
LLM runtime	Ollama + llama3.2:3b	client >=0.3.0
Local database	SQLite	stdlib
Reverse proxy / static server	NGINX	alpine
Runtime	Python / Node	3.11 / 18
Platform	Snowpark Container Services	—
CI/CD	GitHub Actions + Snowflake CLI	—

13.3 Local setup

```
# Backend - terminal 1
cd backend && cp .env.example .env
pip install -r requirements.txt
uvicorn main:app --reload --port 8000          # docs at /docs

# Frontend - terminal 2
cd frontend && cp .env.example .env
npm install && npm start                       # http://localhost:3000

# AI assistant (optional) - terminal 3
ollama pull llama3.2:3b                       # ~2 GB
curl http://localhost:11434/api/tags          # verify
```

13.4 Glossary

Term	Meaning
Gold layer	Curated, analytics-ready star schema produced by the Data Engineering capstone
SPCS	Snowpark Container Services — Snowflake's managed container runtime
Compute pool	The SPCS node pool a service runs on
Type-2 SCD	Slowly changing dimension retaining history via <code>valid_from</code> / <code>valid_to</code> / <code>is_current</code>
Tool calling	LLM pattern where the model selects a typed function instead of generating code or SQL
<code>CLIENT_VALIDATION</code>	Flag selecting mock local auth + CORS (<code>Dev</code>) vs SPCS OAuth (<code>Snowflake</code>)